

Quick Start Guide - AdvReal Training & Inference

TL;DR: Get running in 5 minutes

Ultra Quick Start

1 Install (2 minutes)

```
conda create -n advreal python=3.8.13
conda activate advreal
pip install torch==1.13.1 torchvision==0.14.1 --extra-index-url
https://download.pytorch.org/whl/cu117
pip install pytorch3d==0.6.2
pip install -r requirements.txt
```

2 Download Data (external)

Download from [Google Drive](#) and extract to **data/**

3 Train YOLOv5 Patch (1 command)

```
python train.py \
  --nepoch 800 \
  --save_path results/yolov5_person \
  --arch yolov5 \
  --cfg configs/baseline/v5.yaml \
  --seed_type fixed \
  --loss_type max_iou
```

4 Test with Webcam Demo

```
python scripts/demo.py --cfg baseline/v5.yaml --save_path ./results/demo/
```

Press 'q' to quit

Training All Detectors

Command	Detector	Config
<code>python train.py --arch yolov2 --cfg configs/baseline/v2.yaml --save_path results/yolov2 --nepoch 800</code>	YOLOv2	v2.yaml

Command	Detector	Config
<code>python train.py --arch yolov3 --cfg configs/baseline/v3.yaml --save_path results/yolov3 --nepoch 800</code>	YOLOv3	v3.yaml
<code>python train.py --arch yolov5 --cfg configs/baseline/v5.yaml --save_path results/yolov5 --nepoch 800</code>	YOLOv5	v5.yaml
<code>python train.py --arch yolov11 --cfg configs/baseline/v11.yaml --save_path results/yolov11 --nepoch 800</code>	YOLOv11	v11.yaml
<code>python train.py --arch rcnn --cfg configs/baseline/faster_rcnn.yaml --save_path results/rcnn --nepoch 800</code>	Faster R-CNN	faster_rcnn.yaml
<code>python train.py --arch deformable-detr --cfg configs/baseline/ddetr.yaml --save_path results/ddetr --nepoch 800</code>	D-DETR	ddetr.yaml

Required Directory Structure

AdvReal/	
├─ data/	
│ └─ INRIAPerson/Train/pos/	# Person images
│ └─ background_trans/	# Background images
│ └─ Archive/	# 3D meshes (man.obj, etc.)
├─ configs/baseline/	# Configuration files
├─ detlib/HHDet/	# Detector weights
└─ results/	# Training outputs (auto-created)

Key Parameters

Parameter	What it does	Default	When to change
<code>--nepoch</code>	Training iterations	800	Use 500 for quick test, 1000+ for best results
<code>--lr</code>	Learning rate	0.03	Reduce to 0.01 if loss oscillates
<code>--batch_size</code>	Images per iteration	2	Reduce if GPU OOM
<code>--loss_type</code>	How to compute loss	max_iou	Try max_conf if max_iou fails
<code>--arch</code>	Which detector	yolov2	Match with config file

Monitoring Training

TensorBoard:

```
tensorboard --logdir results/yolov5_person/runs/
```

Check output:

```
results/yolov5_person/
├─ patch_epoch_0.png      # Initial patch
├─ patch_epoch_799.png    # Final patch ← Use this one!
└─ composite/            # Sample renders
```

 Common Issues

Problem	Solution
CUDA OOM	--batch_size 2 or reduce INPUT_SIZE to [320, 320]
Dataset not found	Download data or update IMG_DIR in config
Weights not found	Download model weights (see full guide)
Slow training	Reduce --num_workers or INPUT_SIZE
Loss not decreasing	Increase --lr or check dataset has target class

 Expected Results

After 800 epochs:

- Loss should drop from ~2.0 to ~0.3-0.6
- Patch should show visible patterns
- Attack Success Rate (ASR): 80-92% depending on detector

Training time:

- ~2-4 hours on RTX 3090
- ~5-8 hours on RTX 2080 Ti
- Varies with batch size and image count

 Full Documentation

For detailed explanations, see:

- **Complete guide:** docs/training_and_inference_guide.md
- **Add new detector:** docs/add_new_detector_report.md
- **Change target class:** docs/change_target_object_report.md

 Ready to generate adversarial patches!

